EMPLOYEE MANAGEMENT SYSTEM

3 main tables:

1. **Users**
2. **Departments**
3. **Employees**

Users Table (Authentication)

user_id	username	password	role
1	admin	admin123	admin
2	manager	manager123	employer
3	john	john123	employee

Departments Table

dept_id	dept_name
1	HR
2	IT
3	Finance

Employees Table

emp_id
name
age
salary
dept_id
user_id

Foreign keys:

- **dept_id → Departments.dept_id**
- **user_id → Users.user_id**

1. Project Title

Employee Management System (EMS) – A command-line Python application integrated with SQLite3 database for managing employees, departments, and users with authentication and CRUD operations.

2. Objective

- To apply Python programming skills for building a functional application.
- To integrate Python with SQLite3 database.
- To understand and implement **CRUD operations**: Create, Read, Update, Delete.
- To design a proper **database schema** with multiple related tables.
- To implement user authentication for **admin, employer, and employee roles**.
- To create a **command-line interface (CLI)** for easy interaction.

3. Tools & Technologies

- **Programming Language:** Python 3.x
- **Database:** SQLite3
- **Libraries Used:** sqlite3, os, platform

4. Database Design

4.1 Tables

1. **users** – Stores all system users (admin, employer, employee)
    a. `user_id` INTEGER PRIMARY KEY AUTOINCREMENT
    b. `username` TEXT UNIQUE
    c. `password` TEXT
    d. `role` TEXT (`admin`, `employer`, `employee`)
2. **departments** – Stores department information
    a. `dept_id` INTEGER PRIMARY KEY AUTOINCREMENT
    b. `dept_name` TEXT UNIQUE
3. **employees** – Stores employee details and links to departments and users
    a. `emp_id` INTEGER PRIMARY KEY AUTOINCREMENT
    b. `name` TEXT
    c. `age` INTEGER
    d. `salary` REAL
    e. `dept_id` INTEGER (Foreign key referencing departments)
    f. `user_id` INTEGER (Foreign key referencing users)

4.2 Relationships

- `employees.dept_id → departments.dept_id` (Many-to-One)
- `employees.user_id → users.user_id` (Many-to-One)

5. Features

5.1 User Roles

1. **Admin:**
    a. Add users (admin, employer, employee)
    b. Add and view departments
    c. Add, view, update, delete employees
2. **Employer:**
    a. View all employees
    b. Update employee salaries
3. **Employee:**
    a. View own profile

5.2 Command-Line Interface (CLI)

- Each menu is cleared before display for clarity.
- Menus have headers and separators (`===`) for readability.
- Friendly messages for success, errors, or invalid inputs.
- `pause()` function allows user to read results before returning to menu.

5.3 Login & Authentication

- Users enter **username** and **password**.
- If credentials are invalid, the program gives option to **retry** or **exit**.
- First admin is created during initial setup if no users exist.

5.4 Exception Handling

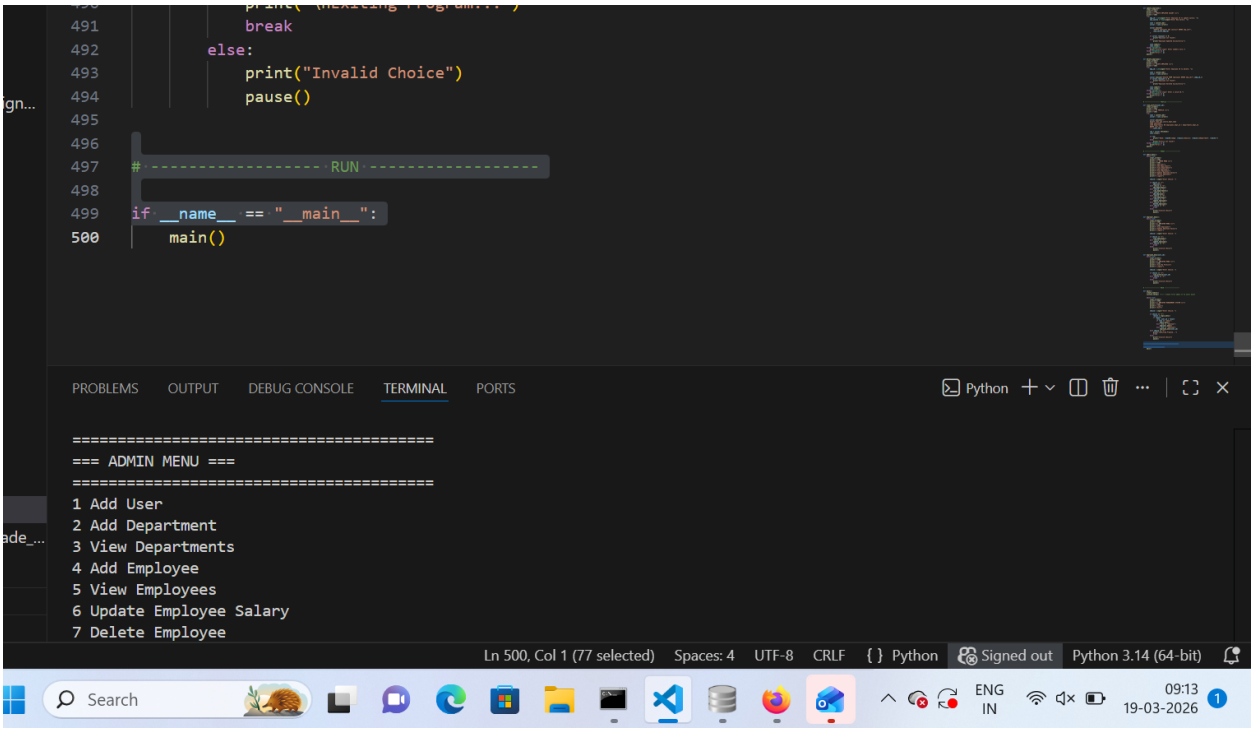
- Handles invalid inputs (non-integer IDs, wrong data types for salary/age).
- Handles database errors like duplicate usernames or departments.
- Gracefully handles empty tables (no employees or departments).

6. Application Flow

1. Program starts and creates database tables.

2. Initial setup checks for existing users; if none, creates admin user.
3. Main menu offers:
    a. Login
    b. Exit
4. After login, user is redirected to the **menu based on role**.
5. Admin/Employer/Employee can perform their respective actions.
6. Program continues until the user chooses to **logout or exit**.

Screenshots:

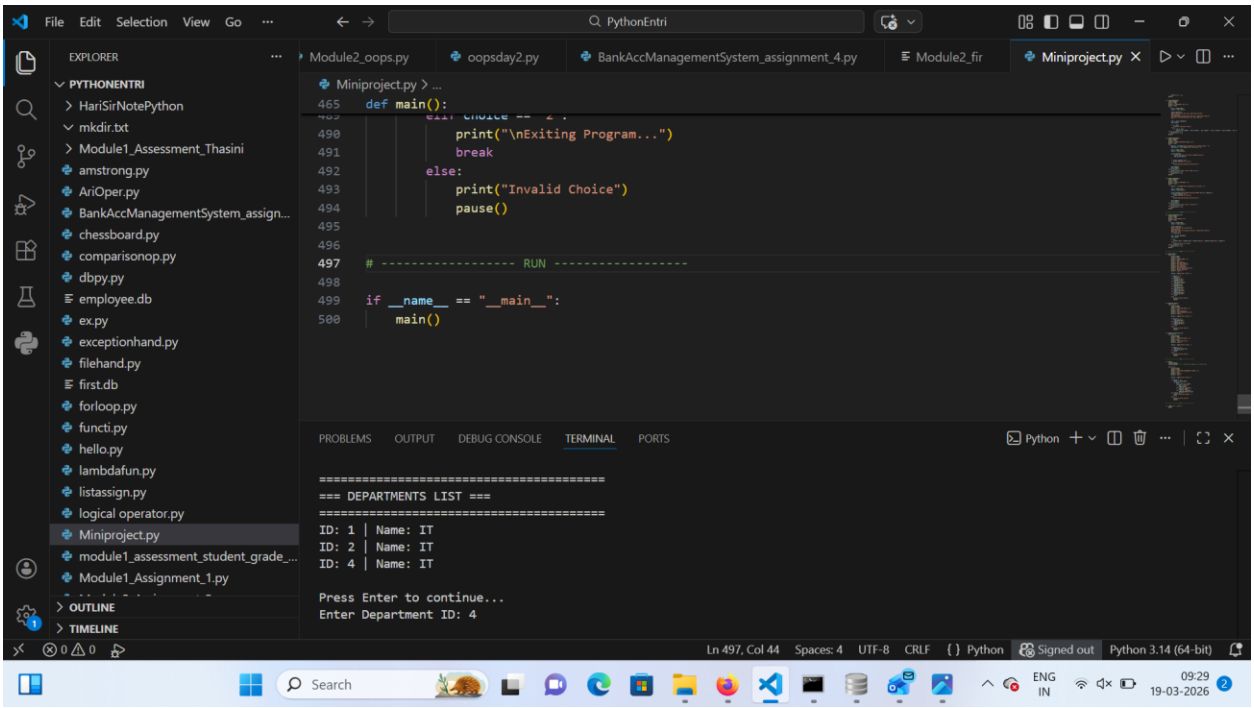


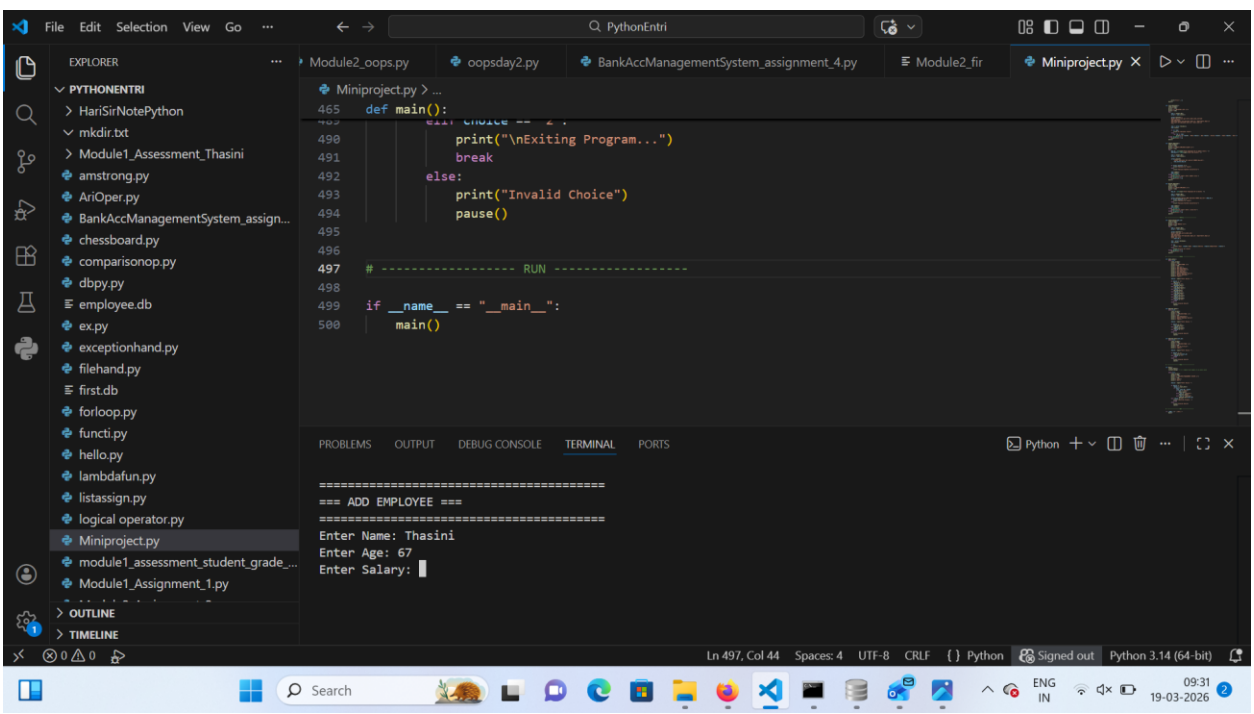
```python
def main():
            print("\nExiting Program...")
            break
        else:
            print("Invalid Choice")
            pause()


# ------------------ RUN ------------------

if __name__ == "__main__":
    main()
```

```
=========================================
=== ADD USER ===
=========================================
Enter username: admin
Enter password: admin123
Enter role (admin/employer/employee): admin
Username already exists!

Press Enter to continue...
```

```python
def main():
            print("\nExiting Program...")
            break
        else:
            print("Invalid Choice")
            pause()


# ------------------ RUN ------------------

if __name__ == "__main__":
    main()
```

```
=========================================
=== LOGIN MENU ===
=========================================
Enter Username: admin
Enter Password: admin123

Login Successful!


Press Enter to continue...
```


File Edit Selection View Go
PythonEntri
EXPLORER
PYTHONENTRI
HariSirNotePython
mkdir.txt
Module1_Assessment_Thasini
amstrong.py
AriOper.py
BankAccManagementSystem_assign...
chessboard.py
comparisonop.py
dbpy.py
employee.db
ex.py
exceptionhand.py
filehand.py
first.db
forloop.py
functi.py
hello.py
lambdafun.py
listassign.py
logical operator.py
Miniproject.py
module1_assessment_student_grade_...
Module1_Assignment_1.py
OUTLINE
TIMELINE
Module2_oops.py
oopsday2.py
BankAccManagementSystem_assignment_4.py
Module2_fir
Miniproject.py
Miniproject.py > ...
465 def main():
490 print("\nExiting Program...")
491 break
492 else:
493 print("Invalid Choice")
494 pause()
497 # ---------- RUN ----------
499 if __name__ == "__main__":
500 main()
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python
==========
=== DEPARTMENTS LIST ===
==========
ID: 1 | Name: IT
ID: 2 | Name: IT
ID: 4 | Name: IT
Press Enter to continue...
Enter Department ID: 4
Ln 497, Col 44 Spaces: 4 UTF-8 CRLF { } Python Signed out Python 3.14 (64-bit)
Search
ENG IN
09:29
19-03-2026


File Edit Selection View Go
PythonEntri
EXPLORER
PYTHONENTRI
HariSirNotePython
mkdir.txt
Module1_Assessment_Thasini
amstrong.py
AriOper.py
BankAccManagementSystem_assign...
chessboard.py
comparisonop.py
dbpy.py
employee.db
ex.py
exceptionhand.py
filehand.py
first.db
forloop.py
functi.py
hello.py
lambdafun.py
listassign.py
logical operator.py
Miniproject.py
module1_assessment_student_grade_...
Module1_Assignment_1.py
OUTLINE
TIMELINE
Module2_oops.py
oopsday2.py
BankAccManagementSystem_assignment_4.py
Module2_fir
Miniproject.py
Miniproject.py > ...
465 def main():
490 print("\nExiting Program...")
491 break
492 else:
493 print("Invalid Choice")
494 pause()
497 # ---------- RUN ----------
499 if __name__ == "__main__":
500 main()
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python
==========
=== ADD EMPLOYEE ===
==========
Enter Name: Thasini
Enter Age: 67
Enter Salary:
Ln 497, Col 44 Spaces: 4 UTF-8 CRLF { } Python Signed out Python 3.14 (64-bit)
Search
ENG IN
09:31
19-03-2026